DSAI LAB PROJECT · MILESTONE 1

ContextGuard

Finding and repairing cell-level errors in tables.
Problem definition, literature review and data audit.

HOSPITAL · THREE ROWS

birmingham	al	35233
birmingham	al	35233
birminghxm	al	35233

Proposed repair: birmingham

Sidhaarth Shree · Jashan · Sahilpreet · Aaradhya

IIT Madras BS in Data Science and Applications · **Group 1**

Submitted **22 October 2026** · github.com/SidhaarthShree07/Group-1-DS-and-AI-Lab-Project

Contents

18,414
verified error cells in the five core tables

81,596
true error cells across the nine audited benchmark tables

5 + 4
unseen test tables and extra training tables audited

0.645
zero-shot score of the chosen base model (± 0.007 over 3 seeds)

1	The problem and why it matters	The problem, with real examples
2	Scope and boundaries	What is in and out of scope
3	Stakeholders	Who needs it
4	Objectives and research questions	Objectives, research questions, evidence
5	Existing landscape: how the problem is addressed today	Existing work, tools and metrics
6	Proposed contribution	What is new and where it lies
7	Data availability	Datasets, audit findings, sufficiency
8	Metrics, baselines and evaluation protocol	Metrics, baselines B1 to B5, protocol
9	Feasibility and plan	Compute, plan, risks
A	Reference material for the cheap detectors (D3–D6)	Cheap detectors D3 to D6
B	Installation check for Raha (head start for Milestone 3)	Raha installation check
C	Team contributions for Milestone 1	Who did what
	References	24 sources

1 The problem and why it matters

Machine-learning models are only as reliable as the tables they are trained on, and real tables contain errors that are invisible when a cell is looked at alone. A cell can be **well-formed and still wrong given the rest of its row**. Three real examples from the benchmark tables we use:

Table	What the cell looks like	Why it is wrong
Hospital, row 4	`City = "birminghxm"`, `ZipCode = 35233`, `State = "al"`	48 rows share zip 35233; 45 say `birmingham` and 3 are typos of it. The zip code and state point to one answer; the cell disagrees
Flights, row 6 (flight AA-204-LAX-MCO)	`sched_arr_time = "12/02/2011 6:55 a.m."` ; the clean table has the bare clock time `6:55 a.m.`	A date has been glued onto a time; the value is valid text but breaks the column's format and cannot be spotted by looking at the cell alone
Beers, row 4	`abv = "0.09%"`, `ounces = "12.0 oz"`	The same quantity is written as `0.09` and `12` in the clean version; formats are mixed inside one column

Rows are counted from 1 in file order.

Existing approaches handle part of this. Rule systems need an expert to write constraints for every dataset. Statistical detectors flag values that are rare, but a rare value is often correct, and a wrong value that looks ordinary is missed. Learning-based cleaners such as Raha and Baran need labelled rows for every new table. None of these reads the row, the column's usual behaviour and the dependencies between columns together, which is what a person does when deciding that `birminghxm` is a typo.

The problem matters because errors in training data propagate into the model: CleanML (Li et al., ICDE 2021) [7] measured the effect of cleaning on downstream classifiers across 14 real datasets and found effects ranging from positive to neutral to negative depending on the dataset and the cleaning method. Cleaning that is applied blindly can therefore hurt, which is why **ContextGuard repairs conservatively and measures the effect on a downstream model**.

Where the problem arises. It appears wherever tables are collected from several sources or typed by people and then used for analysis or model training: hospital and provider records, flight and booking data, product catalogues, bibliographic and address data. In a typical machine-learning pipeline the table is cleaned once, before training, by a data scientist or data engineer. This is the setting ContextGuard targets: offline work on single tables, with a person reviewing what the system is unsure about.

The project. ContextGuard finds cell-level errors in a relational table, names the error type, proposes a repair and attaches a confidence score; only high-confidence repairs are applied automatically and the rest go to a human review list.

2 Scope and boundaries

In scope

- Cell-level error detection, error typing and repair in single relational CSV tables of up to tens of thousands of rows.
- English-language values; the four error types annotated in the TableEG benchmark: missing value, pattern violation, rule violation (dependency between columns) and outlier.
- A confidence gate with separate thresholds for flagging a cell and for applying a repair automatically.
- A demo that takes an uploaded CSV and returns a cleaned file, a change log and a list of cells for review.

Out of scope

- Multi-table joins and entity resolution across tables, free-text documents, images and streaming data.
- Automatic overwriting of data without passing the confidence gate.
- Any claim to have invented LLM-based data cleaning: prior work exists (Section 5.3); our contribution is the specific combination described in Section 6.

3 Stakeholders

Stakeholder	Type	What they need from the system
Data scientists and ML engineers who clean training data	Primary	Fewer wrong labels and features before modelling, without writing rules per dataset
Data engineers and analysts who own data pipelines	Primary	An audit trail: what was changed, why, and with what confidence
Domain data owners (for example hospital records, finance, operations)	Secondary	A short list of uncertain cells to review, with the supporting evidence
People affected by decisions made with models trained on the data (patients, customers, readers of reports)	Secondary	Models trained on fewer wrong records, and repairs that are conservative rather than guessed
The course team	Secondary	A reproducible, honestly evaluated deep-learning / generative-AI project

4 Objectives and research questions

Targets are deliberately stated in general terms. Exact values are set after the first pilot runs on the final pipeline (Milestone 3).

#	Objective	Target	Measured in
O1	Improve on the baselines at cell-level error detection	Better macro-F1 than the baselines on the in-domain test tables	M5
O2	Repair safely	Only a small share of applied repairs are wrong at the chosen confidence threshold	M5
O3	Generalise to unseen tables	On tables never used in training (Restaurant, Marketing, Soccer, Billionaire and a real-error address slice), F1 above a zero-shot LLM and above Raha given 20 labelled rows	M5
O4	Keep LLM use cheap	The cheap candidate stage keeps most true errors while sending only a minority of cells to the LLM	M3–M5
O5	Show downstream value	Report the share of the dirty-versus-clean performance gap closed on a downstream classifier	M5
O6	Deliver an end-to-end tool	A user uploads a CSV and receives the cleaned data plus a change log	M6

#	Research question	Hypothesis	Evidence that answers it
RQ1	Does LLM verification on top of cheap statistical candidates beat rules, statistics, Raha/Baran and a zero-shot LLM at detection?	The fine-tuned hybrid reaches a higher F1 than the baselines, mainly by raising precision	Detection precision, recall, F1 per table for all methods
RQ2	Can the system repair errors safely?	With a confidence threshold, only a small share of applied repairs make a cell worse, at useful coverage	Repair accuracy, false-repair rate, precision-coverage curve
RQ3	Does it generalise to tables it never saw?	F1 drops on unseen tables but stays above Raha with 20 labels and above a zero-shot LLM	In-domain versus unseen-table results
RQ4	Does validated cleaning improve a downstream model?	Training on cleaned data closes a measurable part of the gap between dirty and gold-clean training data	Downstream AUROC and F1 for dirty, cleaned and gold-clean data

What evidence will show that the approach works. (1) Per-table detection precision, recall and F1, and error-type macro-F1, for ContextGuard against baselines B1 to B5, as mean ± standard deviation over seeds (RQ1). (2) A precision–coverage curve and the false-repair rate at the chosen threshold, showing that confident repairs are rarely wrong (RQ2). (3) The same metrics on five tables never used in training, against a zero-shot LLM and against Raha with 20 labels (RQ3). (4) The share of the dirty-versus-clean gap closed on a downstream classifier (RQ4). (5) Candidate recall, the share of cells sent to the LLM and GPU-seconds per 1,000 rows (O4). Section 8 defines each metric, Sections 8.5 to 8.7 add the experiments that isolate each component, the limits of the generalisation claim and the rules for judging mixed results, and the pilot in Section 9 shows that the first steps can already be measured.

5 Existing landscape: how the problem is addressed today

Sections 5.1 and 5.2 summarise eight classical and learning-based systems from their original papers; 5.3 covers LLM-based work, 5.4 commercial and open-source tools, 5.5 the metrics and benchmarks used in this literature, and 5.6 the limitations that remain, and 5.7 what the literature establishes, what is still unresolved and how this project addresses it. Numbers are as reported by each paper; they use different tables, label budgets and metrics, so they are context and not a leaderboard.

5.1 Rule- and constraint-based approaches

1 HoloClean
Rekatsinas et al. (*PVLDB 2017*) [1]

CORE MECHANISM

Probabilistic graphical model (factor graph) combining denial constraints (DCs), external dictionaries, and statistical co-occurrences via Gibbs sampling.

BENCHMARK DATASETS

Hospital (1k×19), Flights (2.4k×6), Food (340k×17), Physicians (2.1M×18)

HEADLINE REPORTED RESULT

Table 2: Hospital Repair F1 = **0.832** ($\tau = 0.5$); Flights Repair F1 = **0.763** ($\tau = 0.3$).

PRIMARY STRENGTH

Unifies heterogeneous signals (logical rules + statistics) in a mathematically sound probabilistic framework.

WEAKNESS VS. CONTEXTGUARD

Requires hand-crafted denial constraints (e.g. 9 DCs on Hospital); fails on unstructured attributes; collapses on datasets without redundant DCs (Beers/Rayyan repair F1 ≈ 0.01 in Baran [5]).

2 NADEEF
Dallachiesa et al. (*SIGMOD 2013*) [2]

CORE MECHANISM

Extensible, declarative rule execution engine separating data quality rules (CFDs, MDs, user UDFs) from violation repair algorithms.

BENCHMARK DATASETS

Two real-life datasets with injected noise: *hosp* (US Hospital Compare, 100K records) and *bus* (UK data.gov.uk, 8 tables joined into 160K tuples)

HEADLINE REPORTED RESULT

Generic rule compilation over relational databases.

PRIMARY STRENGTH

Decouples rule definition from repair logic; supports custom user-defined operators in Java.

WEAKNESS VS. CONTEXTGUARD

All integrity constraints and conditional dependencies must be written manually upfront by domain experts; zero generalization to unseen schemas.

3 HyFD
Papenbrock & Naumann (*SIGMOD 2016*) [3]

CORE MECHANISM

Hybrid functional dependency (FD) discovery: focused sampling of record pairs with dependency induction (finds non-FDs), alternating with direct validation of the candidate FDs on the full data (lattice-style traversal).

BENCHMARK DATASETS

More than 20 real and synthetic datasets (e.g. ncvoter, flight, uniprot, plista, TPC-H lineitem), compared against seven algorithms

HEADLINE REPORTED RESULT

In many cases orders of magnitude faster than the best of seven compared algorithms (row-scalability experiment: more than 20× on ncvoter and more than 416× on uniprot).

PRIMARY STRENGTH

State-of-the-art computational efficiency for discovering exact functional dependencies.

WEAKNESS VS. CONTEXTGUARD

Discovers **strict exact** FDs only. A single dirty or noisy cell invalidates a candidate dependency; cannot discover approximate contextual FDs from dirty data without external tolerance thresholds.

5.2 Machine-learning-based approaches

4 Raha
Mahdavi et al. (*SIGMOD 2019*) [4]

CORE MECHANISM

Ensemble of diverse unsupervised error detection algorithms (dBoost, pattern detectors, rule violations, KB lookup) mapped to feature vectors; semi-supervised classifier trained on ≤ 20 user-labeled tuples.

BENCHMARK DATASETS

Hospital, Flights, Beers, Rayyan, Movies, Tax

HEADLINE REPORTED RESULT

Table 5: Cell-level Detection F1: Hospital = **0.72**, Flights = **0.81**, Beers = **0.99**, Rayyan = **0.79**, Movies = **0.86** (mean of 10 runs).

PRIMARY STRENGTH

Configuration-free detection; reaches high precision with minimal human labeling budget (20 tuples).

WEAKNESS VS. CONTEXTGUARD

Detection only (no repair capability); requires fresh human labels for every new dataset (no zero-shot transfer across unseen tables).

5 Baran
Mahdavi & Abedjan (*PVLDB 2020*) [5]

CORE MECHANISM

Semi-supervised error correction unifying value-based, vicinity-based, and domain-based context models; optional transfer learning from Wikipedia revision logs.

BENCHMARK DATASETS

Hospital, Flights, Beers, Rayyan, Tax, IT, Address

HEADLINE REPORTED RESULT

Table 3 (Perfect ED): Hospital F1 = **0.87**, Flights = **1.00**, Beers = **0.90**.
Table 7 (End-to-End Raha+Baran): Hospital F1 = **0.66**, Flights = **0.66**, Beers = **0.90**, Rayyan = **0.35**, Tax = **0.74**.

PRIMARY STRENGTH

Proposes candidates across three context scopes; reaches convergence with only 20 labeled tuples.

WEAKNESS VS. CONTEXTGUARD

Repair capability degrades drastically when cascaded with imperfect error detection (Rayyan F1 drops from 0.52 to 0.35); lacks calibrated confidence to abort uncertain repairs.

6 BClean
Qin et al. (*ICDE 2024*) [6]

CORE MECHANISM

Bayesian network data cleaning with automatic DAG structure learning from observed data and compensatory scoring for inference.

BENCHMARK DATASETS

Hospital (1,000×15), Flights (2,376×6), Soccer (200,000×10), Beers (2,410×11), Inpatient (4,017×11), Facilities (7,992×11); noise rates about 1–30%

HEADLINE REPORTED RESULT

Table 4 (repair precision/recall/F1, arXiv v1): Inpatient F1 = **0.876** for BClean (0.908 for its BClean-UC and BCleanPI variants); Facilities F1 = **0.807** for BClean-UC and BCleanPI (the basic variant has no entry).

PRIMARY STRENGTH

Completely automated Bayesian network construction without manual prior specification.

WEAKNESS VS. CONTEXTGUARD

High computational inference cost on wide tables; structure learning over noisy data propagates structural DAG estimation errors; lacks semantic world-knowledge.

7 CleanML
Li et al. (*ICDE 2021*) [7]

CORE MECHANISM

Systematic empirical study of the downstream classification impact of cleaning for 5 error types (missing values, outliers, duplicates, inconsistencies, mislabels), with several cleaning methods per type, across 14 real datasets.

BENCHMARK DATASETS

14 real-world ML datasets (Credit, Marketing, Restaurant, etc.) × 7 classifiers

HEADLINE REPORTED RESULT

Concludes that the impact on downstream models is "varying": mostly insignificant for outliers and inconsistencies, mostly positive or insignificant for missing values and mislabels, mostly insignificant or negative for duplicates; it depends strongly on the dataset and the cleaning method (Sec. 6, Table 16).

PRIMARY STRENGTH

Rigorous benchmarking of downstream ML utility rather than isolated cell-level cleaning metrics.

WEAKNESS VS. CONTEXTGUARD

Benchmark framework rather than an automated cleaning tool; uses existing cleaners (mean imputation, SD / IQR / Isolation-Forest outlier detection, HoloClean, ZeroER, OpenRefine, cleanlab) and does not propose a new one.

8 ActiveClean
Krishnan et al. (*PVLDB 2016*) [8]

CORE MECHANISM

Progressive, model-aware data cleaning selecting batches of records to clean based on gradient impact on convex statistical loss functions.

BENCHMARK DATASETS

UCI Adult, UCI EEG, MNIST, IMDB (movie-genre prediction), Dollars For Docs

HEADLINE REPORTED RESULT

Abstract: its optimizations improve model accuracy by up to **2.5×** for the same amount of data cleaned (an ablation of its own optimizations; comparisons with active learning and uniform sampling are reported separately).

PRIMARY STRENGTH

Directly optimizes downstream ML model performance; cleans only data points that influence model gradients.

WEAKNESS VS. CONTEXTGUARD

Constrained strictly to convex loss models (linear/logistic regression, SVM); requires manual human worker to perform actual record edits; inapplicable to complex feature interactions.

5.3 LLM-based approaches

Numbers below are as reported by each paper; they use different splits, label budgets and metrics, so they are context and not a leaderboard.

- **ZeroED** [16] (Ni et al., arXiv 2504.05345, 2025). An LLM labels representative cells from clusters, labels are spread within each cluster, and a classifier then detects errors in the whole table. Reports F1 0.811 on Hospital, 0.722 on Flights and 0.774 on Beers, and up to 90% fewer tokens. *Limitation:* detection only (no repair), no confidence calibration, and the original system relies on a large LLM.
- **Exploring LLM agents for cleaning tabular ML datasets** [17] (Bendinelli et al., ICLR 2025 workshop, arXiv 2503.06664). A frontier LLM writes Python to clean training data and is scored by model performance. *Limitation:* gains were under 1% on one dataset, it struggles with errors that span many rows, and it uses large hosted models.
- **Jellyfish** [18] (Zhang et al., EMNLP 2024). LoRA instruction-tuning of 7–13B local LLMs on four data-preprocessing tasks; Jellyfish-13B reports error-detection F1 95.59 on Hospital, but Hospital was part of its training data, so that number is not an unseen-data result. *Limitation:* error detection evaluated on two datasets only, the LLM is run on every cell, and there is no repair benchmark or calibration.
- **Table-GPT** [19] (Li et al., SIGMOD 2024). "Table-tuning" of GPT-3.5 on synthesised table tasks; zero-shot error-detection F1 0.565 against 0.058 for the untuned model on one benchmark. *Limitation:* proprietary model and not evaluated on the Raha/Baran benchmarks.
- **Cocoon** [20] (Zhang, Huang and Wu, arXiv 2410.15547, 2024). Breaks cleaning into human-like steps, with LLM-derived semantic rules combined with statistics; reports Hospital F1 0.90 (the metric is not clearly defined in the paper). *Limitation:* depends on a hosted frontier model (cost, privacy) and gives no confidence estimates.
- **IterClean** [21] (Ni et al., ACM TURC 2024). An iterative LLM loop of detector, verifier and repairer using five labelled tuples. *Limitation:* a short paper with a thin evaluation and no downstream test.
- **ForestED** [22] (Wang et al., KDD 2026, arXiv 2512.07246). An LLM induces decision trees that are ensembled; reports Hospital F1 0.756 and an average gain of 16.1% F1 over the best baseline. *Limitation:* detection only, and it defaults to a hosted frontier model.
- **TableEG** [23] (Liu et al., arXiv 2507.10934). A LLaMA-3.1-8B model with LoRA that *generates* realistic errors; it supplies the clean/dirty tables and error annotations we use. *Limitation for our purpose:* it generates errors and does not detect or repair them.
- **Survey: Empowering tabular data preparation with language models** [24] (Chen et al., arXiv 2508.01556, 2025) gives the wider map of language-model methods for data preparation; it has no experiments of its own.

5.4 Industry and commercial tools

Descriptions below follow each product's official documentation (checked 19 Oct 2026). In the pages reviewed, checks are rules, metrics or monitors that a person defines, and we found no documented automatic repair of individual cell values.

Open-Source **Great Expectations**

WHAT THE DOCUMENTATION DESCRIBES

Expectation suites with per-column, column-pair and multi-column expectations, plus custom SQL expectations that return the violating rows.

DOCUMENTATION

docs.greatexpectations.io

RELATION TO CONTEXTUAL CELL ERRORS

A person must write every condition; no documented mechanism that learns dependencies or proposes repairs.

Open-Source Amazon Deequ

WHAT THE DOCUMENTATION DESCRIBES

Spark library of declarative data checks; conditions can use Spark SQL across columns (`satisfies`); results can be returned per row; metrics tracked over time with anomaly detection; constraints can be authored or suggested.

DOCUMENTATION

github.com/awslabs/deequ

RELATION TO CONTEXTUAL CELL ERRORS

Constraints are authored or suggested; no documented repair.

Open-Source Pandera

WHAT THE DOCUMENTATION DESCRIBES

Schema validation for dataframes with column checks and dataframe-level (wide) checks over several columns; lazy validation reports the failing checks and values.

DOCUMENTATION

pandera.readthedocs.io

RELATION TO CONTEXTUAL CELL ERRORS

Checks are functions written by the user; no documented repair.

Open-Source TensorFlow Data Validation

WHAT THE DOCUMENTATION DESCRIBES

Schema-based validation of feature statistics (types, domains, ranges, presence) and training-serving skew and drift detection.

DOCUMENTATION

tensorflow.org/tfx/guide/tfdv

RELATION TO CONTEXTUAL CELL ERRORS

No general cross-feature consistency checks are documented.

Open-Source / Commercial Cleanlab

WHAT THE DOCUMENTATION DESCRIBES

Datalab flags label issues, outliers, near-duplicates and rows with missing features.

DOCUMENTATION

docs.cleanlab.ai

RELATION TO CONTEXTUAL CELL ERRORS

Works at the example level, not on individual contextual cell errors; no documented repair.

Cloud Commercial AWS Glue Data Quality

WHAT THE DOCUMENTATION DESCRIBES

DQDL rules include column rules, `CustomSql` , `ColumnCorrelation` , `DatasetMatch` , `ReferentialIntegrity` and `DetectAnomalies` ; row-level results for some rule types; rule recommendation.

DOCUMENTATION

docs.aws.amazon.com

RELATION TO CONTEXTUAL CELL ERRORS

No dedicated functional-dependency rule type; cross-column conditions must be written (for example in SQL); no documented cell-level repair.

Cloud Commercial Google Cloud Dataplex Data Quality (now Knowledge Catalog)

WHAT THE DOCUMENTATION DESCRIBES

Built-in per-column rules plus user-written SQL row-condition and SQL-assertion rules that can span several columns of a record; failing rows can be queried.

DOCUMENTATION

docs.cloud.google.com

RELATION TO CONTEXTUAL CELL ERRORS

Conditions are written by a person; no documented repair.

Enterprise Commercial **Informatica Data Quality**

WHAT THE DOCUMENTATION DESCRIBES
The product page advertises AI-assisted rule generation, profiling and anomaly detection.

DOCUMENTATION
informatica.com

RELATION TO CONTEXTUAL CELL ERRORS
Not tested; public documentation does not say how it handles cross-column cell errors or repair.

Enterprise Commercial **Talend Data Quality**

WHAT THE DOCUMENTATION DESCRIBES
Profiling with regex and SQL patterns and indicators, plus correlation analysis.

DOCUMENTATION
help.qlik.com

RELATION TO CONTEXTUAL CELL ERRORS
Documented checks are patterns and indicators; a mismatched zip/city pair was not tested against the product.

SaaS Commercial **Monte Carlo Data Observability**

WHAT THE DOCUMENTATION DESCRIBES
Automated monitors on freshness, volume and schema, plus metric monitors on field-level statistics with ML thresholds; validation and custom SQL monitors can flag rows.

DOCUMENTATION
docs.getmontecarlo.com

RELATION TO CONTEXTUAL CELL ERRORS
Row-flagging monitors are defined by users; no documented repair.

Contrast with ContextGuard. The rules and monitors above must be authored by a person and detect only what is specified; ContextGuard learns the table's own dependencies and column behaviour and proposes a repair with a calibrated confidence.

5.5 Metrics and benchmarks in existing work

How success is usually measured

- **Detection:** cell-level precision, recall and F1 against the clean table. Raha reports them with 20 labelled tuples, averaged over 10 runs; ZeroED and ForestED report F1 (ZeroED also reports the tokens saved).
- **Repair:** correctly fixed cells divided by all fixed cells (precision) and by all erroneous cells (recall), counted only when the repair equals the clean value. Baran reports this once with perfect detection and once end to end after Raha's detection; HoloClean reports repair precision, recall and F1 and also runtime.
- **Downstream value:** the change in a classifier's accuracy or F1 after cleaning (CleanML, 14 datasets, 7 classifiers) or the model accuracy reached for the same amount of cleaned data (ActiveClean).
- **LLM data-preparation papers:** F1 on error detection (Jellyfish, Table-GPT).

Common benchmarks. Hospital, Flights, Beers, Rayyan, Movies and Tax from the Raha repository; Food and Physicians (HoloClean); Inpatient, Facilities and Soccer (BClean); Billionaire (ZeroED); the 14 datasets of CleanML; and, newly, the real-error address collection Clean Me If You Can. TableEG supplies clean and dirty versions with error types.

What is weak in how success is defined. The papers use different versions of the same tables, different label budgets and different averaging, so numbers cannot be compared directly. Detection and repair are usually scored separately or with perfect detection, calibration of the confidence is almost never measured, and almost no paper

reports unseen-table results and downstream effect together. Section 8 fixes one rule for ground truth, repair and label budgets for all methods.

5.6 Limitations of current approaches

1. **Cost and privacy.** Recent LLM cleaners (Cocoon, ForestED, Table-GPT, the agents study) depend on large hosted models, and ZeroED needs heavy token-saving tricks. Jellyfish shows local LoRA-tuned models are viable but runs the LLM on every cell.
2. **Detection and repair are evaluated apart.** Raha, ZeroED and ForestED only detect. Baran's repair quality drops sharply with imperfect detection (Hospital F1 0.87 with perfect detection against 0.66 end to end with Raha).
3. **Constraints are written by hand.** HoloClean and NADEEF need user-written rules, and HoloClean scores close to 0 F1 on Beers and Rayyan in Baran's comparison. HyFD discovers only exact functional dependencies, which dirty data breaks. Baseline B1 tests whether dependencies mined automatically (TANE) can replace the hand-written rules (Section 8.2).
4. **No calibrated confidence.** None of the LLM-based cleaners above report calibration, so "do not repair when unsure" cannot be measured.
5. **Little downstream evidence.** CleanML and ActiveClean predate LLM cleaners, and most LLM papers stop at cell-level F1.

5.7 What is established, what is still missing, and how this project addresses it

Sections 5.1 to 5.6 describe the approaches one by one. This section compares them by the *kind* of limitation each has, because the limitations are different in nature, and then derives the research gap from them.

Family	What it does well (established)	Where it falls short	Kind of limitation
Rule-based (NADEEF, HoloClean, HyFD)	Precise and explainable when a correct rule exists; HoloClean reaches F1 0.83 on Hospital and 0.76 on Flights on the paper's own versions [1]	An error is found only if someone has written, or the system can mine, a rule that it violates; close to 0 F1 on Beers and Rayyan in Baran's comparison [5]; exact dependencies break on dirty data	**Coverage of expressible rules.** The method cannot judge a value that no rule talks about
Statistical (dBoost, outlier detectors)	Cheap, needs no labels, good at numeric outliers and unusual formats	Treats rare as wrong: a rare value is often correct, and a wrong value that looks ordinary is missed; looks at one column at a time	**Rarity is a poor stand-in for wrongness.** It ignores the context of the row and the other columns
Learning-based (Raha, Baran, ActiveClean)	Combine many weak signals and learn which to trust from about 20 labelled rows; Raha detection F1 0.72 to 0.99 and Baran repair F1 up to 0.90 on Beers [4][5]	A new table needs new labels, and nothing learned transfers to the next table; repair quality drops when detection is imperfect (Hospital 0.87 with perfect detection, 0.66 end to end)	**Per-table supervision.** The cost of labels is paid again for every dataset
LLM-based (Jellyfish, Table-GPT, ZeroED, Cocoon, ForestED)	Understand values and context in language, work with few or no labels; Jellyfish shows that local, LoRA-tuned 7 to 13B models are viable [18]	Large hosted models or one LLM call per cell; mostly detection only; no calibrated confidence; weak use of evidence from other rows; results on different table versions, sometimes with test tables in the training data	**Cost and unmeasured reliability.** It is not known how often the model is confidently wrong

How the limitations differ. Rule-based methods fail where no rule exists, statistical methods fail because they use rarity instead of context, learning-based methods have the signals but need labels for every table, and LLM-based methods understand context but pay for it in cost and do not tell us how far to trust them. The first two lack context; the third needs supervision; the fourth lacks a measure of reliability.

What the studies have already established.

1. Many error types can be found by combining cheap signals and a small number of labels (Raha).
2. Repair quality is limited by detection quality (Baran).
3. Language models can detect and repair data errors, and small local models fine-tuned for data preparation are viable (Jellyfish, Table-GPT).
4. The effect of cleaning on a downstream model is mixed and depends on the dataset and the method (CleanML).

What remains unresolved, and how the project addresses each point

Gap	Open question	How the project addresses it	Measured by
G1 Context	Can a small open model, given evidence from the rest of the table rather than the cell alone, tell a genuine error from an unusual but valid value without labels for the new table?	Prompt with row, column profile, dependency evidence and similar rows; fine-tuning on instruction examples	O1, RQ1; experiments E1 and E2 (Section 8.5)
G2 Reliability	Is the model's confidence good enough to decide which repairs to apply without review? None of the LLM cleaners in our review reports calibration (Section 5.6, item 4)	Calibrated confidence gate with separate thresholds for flagging and for applying a repair	O2, RQ2; precision–coverage curve; experiment E4
G3 Transfer and use	Do the results hold on tables from other domains, and does cleaning help a downstream model?	Test on five unseen tables and on a downstream classifier, reported together	O3, O5, RQ3, RQ4; Section 8.6
G4 Cost against accuracy	How much of the LLM work can candidate filtering remove before real errors are lost?	Cheap candidate stage; candidate recall and cost reported next to accuracy	O4; experiment E3

The gap therefore follows from the comparison above: the rule-based and statistical families lack context (G1), the learning-based family needs labels per table (G3), and the LLM-based family does not measure reliability or the cost–accuracy trade-off (G2, G4).

6 Proposed contribution

Where the contribution lies. The main contribution is performance and robustness: higher detection and repair quality, a repair step that is safe because of the confidence gate, and results on tables the model never saw. Efficiency (a cheap first stage, so the LLM reads only some cells) and usability (a review list and a change log) are supporting dimensions. The deployment context is also new for this task: a small open model running locally on one GPU, without sending data to a hosted service.

Approach in one paragraph. Cheap statistical and dependency-based detectors flag suspicious cells; only those cells are sent, together with their row, a profile of the column and dependency evidence from the rest of the table, to a fine-tuned open language model of about 8 billion parameters (Gemma 4 E4B-it), which answers whether the cell is an error, of which type, and what the repair is; a calibrated confidence gate decides what is applied automatically and what goes to human review. (Training details belong to Milestone 3.)

What is new, stated honestly. Using LLMs to detect data errors is not new (Section 5.3). We claim the combination below, and each part is tied to a measurable objective:

Value	Contribution	Objective
Novelty	A small open model fine-tuned for detection, typing and repair together, with table-level evidence (dependency statistics, column profiles, neighbour rows) in the prompt, aimed at the cross-row weakness reported for LLM cleaning agents	O1, RQ1

Value	Contribution	Objective
Effectiveness	Higher detection F1 than rules, statistics, Raha/Baran and a zero-shot LLM, and a repair that is safe because of the confidence gate	O1, O2, RQ1, RQ2
Efficiency	The cheap stage decides where to look, so only a minority of cells reach the LLM; the model runs locally on one GPU; cost is reported in GPU-seconds per 1,000 rows	O4
Scalability	Cost grows with the number of flagged cells, not the number of cells; the design target is tables of tens of thousands of rows on one GPU, measured in Milestone 5	O4, O6
Measurable added value	Generalisation to unseen tables and the effect on a downstream model are reported together, which most cleaning papers do not do	O3, O5, RQ3, RQ4

6.1 Why the approach is expected to work

Main goal and supporting benefits. The main research goal is to improve the *reliability* of error detection and repair: finding more of the genuine errors and applying few wrong repairs, with a confidence that can be trusted. Lower computational cost (O4) and easier use (O6) are supporting benefits. They matter, but they must not be bought with lower reliability, and the results are judged on the main goal first (Section 8.7).

Each of the three ideas rests on an assumption that the experiments in Section 8.5 are designed to test.

Idea	Why it is expected to help	Assumption behind the expectation	What could go wrong	How we find out
Context in the prompt (row, column profile, dependency evidence, similar rows)	Whether a value is wrong depends on its surroundings. In the Hospital example, `birminghxm` is spotted because the other rows with the same zip code say `birmingham`. The column profile shows what is normal, so an unusual value that fits its dependency group can be told apart from a real error	Errors leave a trace in the table itself (a broken pattern or a broken cross-column consistency), and the dirty table is mostly correct, so its majority values are a useful reference	Tables with little redundancy (Marketing's random category swaps) give little evidence; in heavily corrupted columns the majority itself may be wrong; a dependency can hold by coincidence and mislead	E2: the same model with and without each context block, per table and per error type
Candidate filtering	About 9.5% of the cells in the five core tables are wrong, so reading every cell spends most of the compute on correct values. The cheap detectors (D3 to D6, dependency checks) look at different signals and are inexpensive	Most true errors trip at least one cheap detector, and the detectors miss different errors	An error that looks ordinary is never flagged and never reaches the model, so the filter sets a ceiling on recall for the whole system	E3: filtering alone, full pipeline and the model on all cells; candidate recall per table and type; a random sample of unflagged cells sent to the model to estimate what the filter loses

Idea	Why it is expected to help	Assumption behind the expectation	What could go wrong	How we find out
Fine-tuning	The base model has not seen our four error types, our answer format or what typical errors look like. The zero-shot pilot gave an AUROC of 0.78 to 0.86 but a base-model repair accuracy of only about a third (`outputs/reports/model_bakeoff/README.md`)	Instruction examples from the training tables teach the task and how to read the table evidence, and this transfers to other tables	Overfitting to the error styles of the five training tables; injected errors differ from real ones	E1: the same model, prompt and candidates, before and after fine-tuning; unseen-table results

Why the combination is expected to improve the overall result despite these limits. Filtering costs some recall but removes most correct cells, so the model sees a smaller, harder set and can spend its capacity on the context of each cell. Context helps it separate real errors from unusual but valid values, which is where rules and statistics fail. Fine-tuning teaches the format and the types and gives the confidence that the gate needs. The expectation is not certain, and it has a clear test: the full pipeline must beat the cheap detectors alone, which shows the model adds value, and it should come close to the model on all cells at a fraction of the cost. If the filter loses more errors than the model adds, the candidate set is widened and this is reported as a result, not hidden.

7 Data availability

7.1 Master Dataset Inventory

The table below catalogs the 15 datasets approved for ContextGuard model fine-tuning (Gemma 4 E4B-it), baseline benchmarking, and unseen generalization evaluation. All rows and cell counts for the 9 benchmark tables have been computed directly from the CSV files via `scripts/data_inventory.py` under Raha's ground-truth normalisation rule (Section 7.2). The external tables (Tax, Billionaire, Inpatient, Facilities and the Germany slice of Clean Me If You Can) were computed in the same way by `scripts/external_inventory.py` ; their error types come from a documented heuristic because the sources have no annotations (Section 7.7).

Train + In-Domain **Hospital**

SOURCE LINK / CITATION
Raha Repo / US CMS

FILE VERSION
Canonical Raha `datasets/hospital`

ROWS × DATA COLS (TOTAL COLS)
1,000 × 19 (20)

ERROR CELLS (RAHA RULE)
509

ERROR RATE
2.68%

ERROR TYPES PRESENT
Rule (408), Pattern (101)

LICENCE
Apache-2.0

NOTES
Cell-for-cell match with published literature; strong FDs (zip → city, measure code → name).

Train + In-Domain

Flights

SOURCE LINK / CITATION
Raha Repo / CleanML

FILE VERSION
Canonical Raha `datasets/flights`

ROWS × DATA COLS (TOTAL COLS)
2,376 × 6 (7)

ERROR CELLS (RAHA RULE)
4,920

ERROR RATE
34.51%

ERROR TYPES PRESENT
Missing (2,312), Pattern (1,459), Rule (1,149)

LICENCE
Apache-2.0

NOTES
High error density; 904 unannotated format errors classified via deterministic heuristic.

Train + In-Domain

Beers

SOURCE LINK / CITATION
Raha Repo

FILE VERSION
Canonical Raha `datasets/beers`

ROWS × DATA COLS (TOTAL COLS)
2,410 × 10 (11)

ERROR CELLS (RAHA RULE)
4,362

ERROR RATE
18.10%

ERROR TYPES PRESENT
Pattern (3,077), Missing (1,132), Rule (153)

LICENCE
Apache-2.0

NOTES
Recovered 1,005 errors dropped by TableEG's copy; ABV/IBU formatting and outliers.

Train + In-Domain

Rayyan

SOURCE LINK / CITATION
Raha Repo

FILE VERSION
Canonical Raha `datasets/rayyan`

ROWS × DATA COLS (TOTAL COLS)
1,000 × 11 (11)

ERROR CELLS (RAHA RULE)
948

ERROR RATE
8.62%

ERROR TYPES PRESENT
Rule (619), Pattern (254), Missing (75)

LICENCE
Apache-2.0

NOTES
Medical citations; exact annotation mapping: 619 rule, 254 pattern, 75 missing.

Train + In-Domain

Movies

SOURCE LINK / CITATION
Raha Repo

FILE VERSION
Canonical Raha `datasets/movies_1`

ROWS × DATA COLS (TOTAL COLS)
7,390 × 17 (17)

ERROR CELLS (RAHA RULE)
7,675

ERROR RATE
6.11%

ERROR TYPES PRESENT
Pattern (7,568), Missing (107)

LICENCE
Apache-2.0

NOTES
Recovered a net 668 errors lost in TableEG, where `RatingValue` was re-formatted (`7.0` became `7`); all 7,675 annotated.

Train + In-Domain

Tax

SOURCE LINK / CITATION
Raha Repo

FILE VERSION
Raha `datasets/tax` (full table; a 20,000-row sample is drawn in M2)

ROWS × DATA COLS (TOTAL COLS)
200,000 × 15 (15)

ERROR CELLS (RAHA RULE)
121,219

ERROR RATE
4.04%

ERROR TYPES PRESENT
Pattern (2,166), Rule (800), Format artifact (118,253)

LICENCE
Apache-2.0

NOTES
97.6% of the error cells are re-formatting: `rate` `3` vs `3.0` (87,342) and `zip` without vs with a leading zero (31,311). Only 2,966 cells are injected errors (typos, `-*` suffixes, swaps).

Train-Only

Inpatient

SOURCE LINK / CITATION
BClean Repo

FILE VERSION
`dataset/Inpatient/Inpatient_dirty_10.csv` (10% variant)

ROWS × DATA COLS (TOTAL COLS)
4,017 × 11 (11)

ERROR CELLS (RAHA RULE)
4,465

ERROR RATE
10.10%

ERROR TYPES PRESENT
Missing (1,800), Pattern (1,684), Rule (981)

LICENCE
None stated

NOTES
Dirty file has an extra index column (dropped after validation); 4 duplicate rows in the clean file. Same domain as Hospital, so training only.

Train-Only

Facilities

SOURCE LINK / CITATION
BClean Repo

FILE VERSION
`dataset/facilities/facilities_dirty10.csv` (10% variant)

ROWS × DATA COLS (TOTAL COLS)
7,992 × 10 (10)

ERROR CELLS (RAHA RULE)
8,318

ERROR RATE
10.41%

ERROR TYPES PRESENT
Missing (3,001), Pattern (2,384), Rule (2,933)

LICENCE
None stated

NOTES
Same index-column handling; BClean's own example script uses the 5% file. Training only.

Unseen-Test

Restaurant

SOURCE LINK / CITATION
TableEG / CleanML

FILE VERSION
TableEG `source/Restaurant`

ROWS × DATA COLS (TOTAL COLS)
12,007 × 11 (12)

ERROR CELLS (RAHA RULE)
633

ERROR RATE
0.48%

ERROR TYPES PRESENT
Pattern (633)

LICENCE
None stated

NOTES
Dynamically computed from real clean CSV: categories (395), name (121), neighborhood (81), streetAddress (35), city (1).

Unseen-Test Marketing

SOURCE LINK / CITATION
TableEG / CleanML

FILE VERSION
TableEG `source/Marketing`

ROWS × DATA COLS (TOTAL COLS)
8,993 × 14 (15)

ERROR CELLS (RAHA RULE)
26,815

ERROR RATE
21.30%

ERROR TYPES PRESENT
Pattern (24,121), Missing (2,694)

LICENCE
None stated

NOTES
Category value swaps (`Male` ↔ `Female` , age buckets); evaluate cross-domain zero-shot.

Unseen-Test Soccer

SOURCE LINK / CITATION
TableEG `test_dataset/exp_1`

FILE VERSION
`BART_output/soccer/soccer_10`

ROWS × DATA COLS (TOTAL COLS)
1,999 × 10 (11)

ERROR CELLS (RAHA RULE)
796

ERROR RATE
3.98%

ERROR TYPES PRESENT
Pattern (597), Outliers (199)

LICENCE
None stated

NOTES
Dynamically classified: 199 numeric outliers (`birthyear`), 597 text pattern violations; annotations: none (BART log has no types).

Unseen-Test Billionaire

SOURCE LINK / CITATION
ZeroED Repo

FILE VERSION
`data/billionaire_clean.csv` , `billionaire_error-01.csv`

ROWS × DATA COLS (TOTAL COLS)
2,614 × 22 (22)

ERROR CELLS (RAHA RULE)
5,195

ERROR RATE
9.03%

ERROR TYPES PRESENT
Missing (1,386), Pattern (3,049), Rule (742), Format artifact (18)

LICENCE
None stated

NOTES
The clean file holds 200 `nan (wait to be cleaned)` placeholders. Plain string comparison gives 5,266; 71 cells differ only in whitespace or HTML entities.

Unseen-Test Clean Me If You Can (Germany slice)

SOURCE LINK / CITATION
Zenodo AddressTable v1.0

FILE VERSION
`de_slice/named/slice_de_{clean,dirty}.csv` ; 5,000 rows, seed 42

ROWS × DATA COLS (TOTAL COLS)
5,000 × 9 (10; full slice 374,533 rows)

ERROR CELLS (RAHA RULE)
18,678

ERROR RATE
41.51%

ERROR TYPES PRESENT
Missing (12,087), Pattern (6,292), Rule (295), Format artifact (4)

LICENCE
ODbL 1.0 (OpenStreetMap-derived ground truth); Zenodo record lists CC BY 4.0; code MIT

NOTES
Real web-extracted addresses; every sampled row has an error. Share-alike and attribution apply.

Downstream Test **Credit (RQ4)**

SOURCE LINK / CITATION
TableEG / Kaggle

FILE VERSION
TableEG `source/Credit`

ROWS × DATA COLS (TOTAL COLS)
150,000 × 11 (12)

ERROR CELLS (RAHA RULE)
350,404 (strict) / 38,936 (lenient)

ERROR RATE
21.24% (strict) / 2.36% (lenient)

ERROR TYPES PRESENT
Missing (33,655), Outliers (1,283), Format Artifact (315,466)

LICENCE
Kaggle Competition Rules

NOTES
Target `SeriousDlqin2yrs` (6.68% pos); 17,435 imputed `MonthlyIncome` values (88.44070676); 315,466 float re-saving format artifacts isolated.

Augmentation **Adult, Bike Sharing, Wine Quality**

SOURCE LINK / CITATION
UCI ML Repository

FILE VERSION
`adult.data` 32,561 × 15 (`adult.test` 16,281 stays held out); `day.csv` 731 × 16, `hour.csv` 17,379 × 17; red 1,599 × 12, white 4,898 × 12

ROWS × DATA COLS (TOTAL COLS)
see file version

ERROR CELLS (RAHA RULE)
Synthetic injection

ERROR RATE
≤ 20% of training

ERROR TYPES PRESENT
Synthetic rule and pattern errors

LICENCE
CC BY 4.0

NOTES
Clean tables only; injected with the TableEG LoRA generator and the BigDaMa error generator; attribution kept.

7.2 Ground-Truth Normalisation Rule (Raha Rule)

To ensure direct academic comparability with published baselines (Raha [SIGMOD 2019], Baran [VLDB 2020], HoloClean [VLDB 2017]), ContextGuard adopts Raha's canonical cell comparison rule.

Plain-Language Definition

A tabular cell at row r and column c is defined as an error if and only if its dirty string representation does not match its clean ground-truth representation after deterministic normalisation:

$$\text{CellError}(r, c) \iff \text{Norm}_{\text{Raha}}(\text{Dirty}[r, c]) \neq \text{Norm}_{\text{Raha}}(\text{Clean}[r, c])$$

Normalisation Sequence

Raha's own rule has three steps, applied in this order to both values:

1. **HTML entity unescaping:** escaped entities (e.g. `&` → `&`, `'` → `'`) are unescaped with `html.unescape`.
2. **Whitespace collapsing:** runs of tab, newline and space become a single space (`[\t\n ]+` → `" "`).
3. **Trimming:** tab, newline and space are stripped from both ends (`.strip("\t\n ")`).

`None` is read as the empty string, and literal strings such as `"nan"`, `"null"` or `"N/A"` are kept as they are (so Beers' `"N/A"` against a blank counts as an error). Index and identification columns (`row_id`, `tuple_id`, `index`) are excluded from error counting and from rate denominators. The unification of missing-value tokens belongs to the lenient diagnostic below only.

Strict vs. Lenient Evaluation

- **Strict Evaluation (Canonical):** Checks exact character equality on the normalised strings. Used for all core detection, precision, recall, and repair accuracy metrics.
- **Lenient Evaluation (Diagnostic Only):** In tables where re-saving introduced floating-point precision differences (e.g., Credit `45.0` vs `45`, or rounding to 4 decimals), numbers are parsed as IEEE-754 floats and missing indicators (`"N/A"`, `"?"`, `"-"`) are equated; this is a diagnostic only and does not change any count in the tables. Lenient counting reveals that Credit's true semantic error pool is 38,936 cells (with 34,938 annotated true errors), whereas 315,466 cells are benign float re-saving format artifacts.

7.3 Data Audit Findings & Quality Issues Catalogue

Our physical data audit executed via `scripts/data_inventory.py` uncovered several critical discrepancies across benchmark distributions:

1. **TableEG Data Loss in Beers (1,005 Errors Lost):**
    - *Finding:* TableEG's packaged `beers/dirty.csv` contains only 3,357 error cells, missing 1,005 real errors present in the original Raha benchmark (4,362 errors).
    - *Root Cause:* TableEG's preprocessing script silently converted `"N/A"` string values in the `ibu` (bitterness) column into empty blank cells, deleting 1,005 legitimate missing-value errors.
    - *Resolution:* ContextGuard discards TableEG's copy and establishes the canonical Raha repository file (`raha-master/datasets/beers/`) as our immutable ground truth.
2. **TableEG Data Loss in Movies (a Net 668 Errors Lost):**
    - *Finding:* TableEG's movie copy recorded only 7,007 errors compared to 7,675 in Raha's original `movies_1` table (a net loss of 668 errors).
    - *Root Cause:* Number re-formatting in `RatingValue`: Raha's dirty file has `7.0` where the clean file has `7`, while TableEG's copy has `7` in both, so those cells stop being errors. The cause is not comma formatting (both copies hold the same 5,505 comma values).
    - *Resolution:* ContextGuard utilizes the Raha master copy (`raha-master/datasets/movies_1/`), recovering all 7,675 errors which map 100% to TableEG's annotation catalogue.
3. **Unannotated Changes in Flights (904 Cells):**
    - *Finding:* `flights/dirty.csv` contains 4,920 changed cells under Raha's rule, but TableEG's `flight_annotation.csv` provides only 4,016 annotations.
    - *Root Cause:* 904 time values (`act_dep_time`, `act_arr_time`, `sched_arr_time`, `sched_dep_time`) were modified in the dirty table but omitted from the error log.
    - *Resolution:* ContextGuard applies a deterministic heuristic: all 904 unannotated cells contain time format variations and are categorized as `pattern_violation`.
4. **Unannotated Cells in Beers (1,005 Cells):**
    - *Finding:* TableEG annotated 3,357 cells across `city`, `ounces`, `abv`, and `state`, but left all 1,005 `ibu` errors unannotated.
    - *Resolution:* All 1,005 unannotated cells have `"N/A"` in dirty vs `""` in clean, deterministically classified as `missing_value`.
5. **Clean/Dirty Column Header Mismatches (Beers, Movies, Hospital, Soccer):**

- *Finding:* Header names differ between clean and dirty files:
  - *Hospital (19 diffs):* Clean uses PascalCase (`ProviderNumber`, `HospitalName`), dirty uses snake_case (`provider_number`, `name`).
  - *Beers (2 diffs):* Clean uses hyphens (`beer-name`, `brewery-name`), dirty uses underscores (`beer_name`, `brewery_name`).
  - *Movies (17 diffs):* Clean uses capitalized/spaced headers (`Release Date`, `Cast`, `RatingValue`), dirty uses lowercase snake_case (`release_date`, `full_cast`, `rating_value`).
  - *Soccer (2 diffs):* Clean has `birthyear (integer)` and `season (integer)`, dirty has `birthyear(long)` and `season(long)`.
- *Impact:* Keying joins by column name causes silent data corruption or column drops.
- *Resolution:* All table auditing, training loaders, and evaluation scripts strictly align columns by **positional index** rather than string name matching.

6. **Clean-Side Quality Deficiencies (Gold Labels Not 100% Clean):**
   - *Finding:* "Clean" ground-truth tables contain substantial remaining missing/null values:
     - *Beers:* 1,072 missing cells in `clean.csv` (`ibu`: 1,005, `abv`: 62, `style`: 5).
     - *Movies:* 2,680 missing cells in `clean.csv` (`Filming Locations`: 1,046, `ReviewCount`: 356, `Duration`: 302, `RatingValue`: 232).
     - *Rayyan:* 1,609 missing cells in `clean.csv` (`jounral_abbreviation`: 529, `article_language`: 358, `journal_issn`: 316).
     - *Marketing:* 5,724 missing cells in `Person under 18`.
     - *Duplicate Rows:* Exactly 0 duplicate rows detected across all 9 audited tables.
   - *Significance:* Missing values in clean ground-truth tables cap achievable repair recall; a cell with missing ground truth cannot be supervised as a correction target.

7. **Credit Dataset Origin & Imputation Artifacts:**
   - *Finding:* TableEG's Credit dataset is Kaggle's *"Give Me Some Credit"* ($150,000 \times 11$), not the UCI Default of Credit Card Clients table.
   - *Class Balance:* Highly imbalanced target `SeriousDlqin2yrs`: Class 0 = 139,974 (93.316%), Class 1 = 10,026 (6.684%).
   - *Imputation Artifact:* In `clean.csv`, missing values in `MonthlyIncome` were filled with synthetic imputed numbers: the exact float `88.44070676` repeats **17,435 times**.
   - *Float Format Artifacts:* 315,466 cells differ purely due to integer vs float re-saving format discrepancies (e.g. `45.0` vs `45`). These are isolated in `format_artifact` and excluded from true error metrics.
   - *Resolution:* In RQ4 downstream testing, Credit gold repairs must be evaluated with explicit awareness that ground-truth salaries represent imputed estimates rather than measured true incomes.

8. **Soccer BART Error Injection Verification:**
   - *Finding:* `test_dataset/exp_1/BART_output/soccer/soccer_10/dirty_data.csv` contains exactly **796 cell differences** against `clean_soccer.csv`.
   - *Dynamic Classification:* 199 numeric changes in `birthyear (integer)` are classified as `outliers`; 597 text changes in `stadium`, `team`, and `birthplace` are classified as `pattern_violation`.

- *Verification:* Matches `soccer_10/cellChanges.csv` (796 rows) with **0 mismatches** (1-based index offset $r-1$ resolved). The root BART generation log `error_log.jsonl` contains 1,996 total injected errors. Annotations recorded as `"none (BART log has no types)"`.

9. **Dropped Dataset (University):**
   - *Finding:* Preliminary project drafts referenced a `University` (286×17) held-out benchmark.
   - *Audit Result:* `University` was completely omitted from the official TableEG public release.
   - *Resolution:* University is officially dropped; our unseen-table evaluation is supported by Restaurant, Marketing, Soccer, Billionaire, and Clean Me If You Can.

7.4 Data Hygiene & Leakage Prevention Rules

To guarantee scientific validity and prevent optimistic evaluation bias, ContextGuard enforces four strict data hygiene boundaries:

1. **Leave-Table-Out Generalisation Boundary (RQ3):**
   - Unseen-test tables (`Restaurant`, `Marketing`, `Soccer`, `Billionaire`, `Clean Me If You Can`) are **strictly held out**.
   - They are never used for LoRA fine-tuning, training-set augmentation, column profiling, functional dependency discovery, or in-context few-shot examples.
2. **Deterministic Row-Level Partitioning:**
   - Training tables are partitioned into **70% Train / 15% Validation / 15% Test** splits using a fixed random seed (`seed = 42`).
   - Duplicate row contents are assigned to the same partition to eliminate row-level leakage.
   - Context prompt neighbor rows are retrieved strictly from within the current partition or the train split.
3. **Same-Domain Isolation Caution:**
   - `Inpatient` and `Facilities` share the healthcare administrative domain with `Hospital`.
   - Consequently, `Inpatient` and `Facilities` are classified strictly as **training-only** datasets. They will never be evaluated as "unseen tables" to prevent cross-table domain leakage from inflating generalization metrics. An exact-match check against Hospital found no shared identifiers (no provider numbers or phone numbers), only 1 of Hospital's 45 hospital names and addresses in Facilities and 6 of 44 zip codes; Billionaire shares no rows with any other table (download log, Section 7).
4. **Exclusion of Pre-Packaged Training Triplets:**
   - TableEG ships with pre-generated training files (`dataset/train/*.jsonl`) encompassing 29,527 examples.
   - Audit of these files revealed that they include instances synthesized from `Marketing` and `Restaurant` (e.g., 630 restaurant instances in `train_ErrorCorrection.jsonl`).
   - *Rule:* ContextGuard completely excludes TableEG's `dataset/train/*.jsonl` files from training. All training triplets are synthesized in-house from approved training tables only.
   - *Rule:* Jellyfish-Instruct is excluded because its training set contains Hospital.

7.5 Licensing, Governance, and Attribution

ContextGuard enforces strict adherence to data copyright, licensing restrictions, and provenance attribution:

- **Apache-2.0 Datasets:** The 5 core Raha benchmark tables (`Hospital`, `Flights`, `Beers`, `Rayyan`, `Movies`) and `Tax` are distributed under the permissive Apache-2.0 licence by the BigDaMa laboratory. They are fully

compatible with academic reproduction and open-source benchmarking.

- **Kaggle Competition Rules:** The Credit dataset derives from Kaggle's *"Give Me Some Credit"* competition. It is restricted to non-commercial academic research and benchmarking.
- **No licence stated:** the BClean repository (Inpatient, Facilities) and the ZeroED repository (Billionaire) have no LICENSE file, so we cite and use them for research only and never redistribute them. Note that BClean's README names a different repository, `thethe-github/BClean`, as the home of the paper's code.
- **Open Data Commons Open Database License (ODbL 1.0):** The Germany (`DE`) address slice extracted from *Clean Me If You Can* (Ahmadi et al., PVLDB 2026; Zenodo DOI 10.5281/zenodo.20841898) derives from OpenStreetMap data; the Zenodo record itself lists CC BY 4.0 and the code is MIT, and we follow the stricter ODbL reading. Under ODbL terms, any derivative database produced using this slice must acknowledge OpenStreetMap contributors and remain available under ODbL share-alike provisions.
- **Raw Data Repository Policy:** In accordance with course guidelines and dataset terms, raw CSV files are excluded from git tracking via `.gitignore`. The repository version-controls audit and ingestion scripts (`scripts/data_inventory.py`), schema definitions, and summary tables.

7.6 Data-Sufficiency Argument (Approval Note Response)

The course approval note asked: *"What data is available (sources, quantity, quality), and is it sufficient to train and evaluate reliably?"*

Training Quantity and Representation Sufficiency

ContextGuard's approved training pool spans 4 distinct domains (Healthcare, Aviation, Manufacturing/Consumer Goods, and Academic Publishing) across 5 core benchmark tables plus `Tax`, `Inpatient`, and `Facilities`.

Across the 5 canonical tables alone, we have audited **18,414 verified error cells**: * `Hospital`: 509 errors (2.68% error rate) * `Flights`: 4,920 errors (34.51% error rate) * `Beers`: 4,362 errors (18.10% error rate) * `Rayyan`: 948 errors (8.62% error rate) * `Movies`: 7,675 errors (6.11% error rate)

Applying our 70% training row split and pairing error cells with negative examples (correct cells) yields an estimated **25,000 to 35,000 training candidate triplets**. This comfortably satisfies our budget of 15,000–40,000 examples required for parameter-efficient LoRA fine-tuning of **Gemma 4 E4B-it** (rank 16, bf16).

Error Type Diversity & Imbalance Mitigation

Across the 81,596 true error cells audited in our benchmark corpus (excluding Credit's 315,466 float format artifacts), the four canonical error categories exhibit the following verified distribution:

1. **`missing_value` (39,975 cells, ~48.99%):** Dominant error category; empty strings and placeholder tokens (`"N/A"`, `"Not Available"`) across Credit (33,655), Marketing (2,694), Flights (2,312), Beers (1,132), Movies (107), and Rayyan (75).
2. **`pattern_violation` (37,810 cells, ~46.34%):** Heavily represented via format shifts, typos, and string distortions across Marketing (24,121), Movies (7,568), Beers (3,077), Flights (1,459), Restaurant (633), Soccer (597), Rayyan (254), and Hospital (101).
3. **`rule_violation` (2,329 cells, ~2.85%):** Concentrated in Flights (1,149), Rayyan (619), Hospital (408), and Beers (153).
4. **`outliers` (1,482 cells, ~1.82%):** Concentrated in Credit financial ratios (1,283) and Soccer player birthyears (199).

> **IMPORTANT** **Data Sufficiency Risk & Mitigation:** Natural rule violations are relatively scarce outside Flights, Rayyan, and Hospital. To ensure our fine-tuned model (Gemma 4 E4B-it) generalizes well to semantic constraints, we will augment the training set with synthetic functional dependency violations generated on the clean UCI tables (Adult, Bike Sharing, Wine Quality) using the BigDaMa error generator and TableEG LoRA generator ($\leq 20\%$ of training volume).

Evaluation Stability on Unseen Tables (RQ3)

Our held-out evaluation tables provide sufficient error volume to measure statistically stable, leak-free precision, recall, and F1 scores: * **Restaurant ($12,007 \times 11$):** Contains 633 errors under Raha's rule (TableEG lists 635 annotations; 2 `streetAddress` cells differ only by a double space in the clean file, e.g. `Ste 290` against `Ste 290`, which the rule collapses; all `pattern_violation`: categories 395, name 121, neighborhood 81, streetAddress 35, city 1; 0 errors in priceRange and zipCode). At 0.48% error rate, this provides an ideal testbed for evaluating the model's false-positive rate and false-repair rate. * **Marketing ($8,993 \times 14$):** Contains 26,815 dense categorical errors (21.30% error rate), testing classification and repair of arbitrary semantic swaps. * **Soccer ($1,999 \times 10$):** Contains 796 verified BART injection errors dynamically typed into 199 numeric outliers and 597 text pattern violations, ensuring exact comparison against synthetic baseline benchmarks. * **Downstream Credit ($150,000 \times 11$):** Provides 38,936 semantic differences under the lenient rule (4-decimal rounding; 34,938 of them annotated as true errors) across 150,000 rows to measure downstream AUROC impact on `SeriousDlqin2yrs` (RQ4), with float format artifacts cleanly isolated.

7.7 External Tables: Tax, Inpatient, Facilities, Billionaire and the Germany Slice

These five tables come from the Raha, BClean, ZeroED and Clean Me If You Can repositories (sources, commits, checksums and licences are in `data/download_log.md`). They carry no TableEG-style annotations, so changed cells are typed by a documented heuristic, applied by `scripts/external_inventory.py`:

Type	Rule
`missing_value`	The dirty value is empty or a missing token (`nan`, `null`, `n/a`, `?`, `-`) and the clean value is not
`format_artifact`	The same number written differently (`3` vs `3.0`), or a leading zero added or dropped (`1203` vs `01203`)
`rule_violation`	The dirty value is another valid value of the same clean column: a swap that breaks dependencies on other columns
`pattern_violation`	Any other change: typos, distortions, new strings, other formats

Outliers cannot be told apart from the other types by this method and are not assigned for these tables; they come from Credit and Soccer. The rule is a heuristic, not a verified label of how each error was generated.

Findings

1. **Tax is mostly re-formatting.** Of 121,219 error cells under Raha's rule (4.04%, as in the Raha paper), 118,253 (97.6%) are `rate` written `3.0` instead of `3` and `zip` with a leading zero. Only 2,966 cells (0.1% of all cells) are injected errors. Raha's rule counts all of them, so Tax baseline numbers stay comparable. For training they are kept out of the type totals and out of the positive examples, exactly like Credit's format artifacts; otherwise a 20,000-row sample would teach the model that `3` to `3.0` is an error.

2. **Billionaire's clean file is not fully clean:** 200 cells hold the text `nan (wait to be cleaned)`. Its error count is 5,195 under Raha's rule; plain string comparison would give 5,266.
3. **BClean files:** the dirty files carry an extra unnamed row-index column (dropped after checking that it holds 0, 1, 2, and so on); the Inpatient clean file has 4 duplicate rows. The 10% variants measure 10.10% (Inpatient) and 10.41% (Facilities) changed cells.
4. **Germany slice:** the slice is already published in the Zenodo archive (374,533 rows). The 5,000-row, seed-42 sample has 18,678 error cells (41.51%), 65% of them missing values, because the dirty side is web-extracted text and the clean side is geocoder output. It is a much harder and noisier test than the benchmark tables.
5. **Clean-side placeholders:** Tax's clean file has 206 `nan` cells and Facilities 27 placeholder tokens; none of the five has blank cells (`docs/tables/external_clean_quality.csv`).

Contribution to training and test volume. Excluding format artifacts, the training-only external tables add 15,749 error cells (Tax 2,966, Inpatient 4,465, Facilities 8,318) to the 18,414 of the five core tables. The unseen-test additions are Billionaire (5,177 error cells excluding artifacts) and the Germany sample (18,674).

Decision. Tax's format artifacts are excluded from the training positives and from the type totals; they stay in the Raha-rule counts so baseline numbers remain comparable.

7.8 What the labels establish, and their limits

What the labels establish. A cell is labelled an error when its dirty value differs from the clean value after the normalisation of Section 7.2. This gives one consistent, repeatable rule that is comparable with the published baselines. It establishes that a dirty value *disagrees with a reference table*. It does not establish that the dirty value is wrong in a real-world sense, or that the clean value is the only valid one.

Limitation	Evidence from the audit	How it is handled	Effect on the conclusions
Format-only differences (`3` against `3.0`, leading zeros, `45.0` against `45`)	Tax: 118,253 of 121,219 error cells; Credit: 315,466 cells	Typed `format_artifact`; counted under the Raha rule so baseline numbers stay comparable, but excluded from training positives and from the type totals; strict and lenient counts are both reported	Conclusions about error detection use the cells without artifacts
The normalisation rule decides what counts	Restaurant: 2 cells differ only by a double space, so 633 errors under the rule against 635 annotations	One rule for all methods; whitespace is collapsed, case and punctuation are kept strict; the strict and lenient counts are reported where they differ	A different rule would change the counts, so numbers are valid only under this rule
Incomplete or imputed clean files	Missing cells in the clean files: Beers 1,072, Movies 2,680, Rayyan 1,609; Credit's clean income repeats the value 88.44070676 17,435 times	The cap on achievable recall is stated; the downstream test is repeated on Beers; Credit repairs are read as agreement with an imputation	Repair accuracy on these tables measures agreement with the reference, not truth
Incomplete annotations	Flights: 904 changed cells have no entry in the annotation file; the external tables ship no error types	Unannotated Flights cells are typed `pattern_violation` by a documented rule; external tables are typed by a documented heuristic and labelled as such; type accuracy is computed only on annotated cells	Error-type results are reliable only for annotated cells
A differing value can still be valid	A repair that is a valid alternative (another accepted spelling or format) but not identical to the clean value counts as wrong under exact match	Exact match stays the primary metric because it is comparable with the papers; in Milestone 5 a random sample of false repairs is reviewed by hand and classified as wrong, valid alternative or unclear	The reported false-repair rate is an upper bound on real harm; the reviewed share shows how large the gap is
Injected and real errors differ	Some tables contain injected errors (Soccer was generated with the BART tool); the Germany address slice has real errors	Results are reported per table, so injected and real errors can be told apart	Strong results on injected errors must not be read as strong results on real data

What this means for the conclusions. The results will show how well the system reproduces the reference corrections on these tables under one fixed rule. They will not show that it finds every kind of genuine semantic error in real-world data, and the final report will state them that way. The manual review of false repairs gives a first estimate of how far the reference differs from real-world correctness.

8 Metrics, baselines and evaluation protocol

8.1 Metric definitions

All metrics are computed at the cell level, per table and as a macro-average over tables. Ground truth is Raha's rule: a cell is an error when its dirty value differs from its clean value after unescaping HTML, collapsing whitespace and stripping; the clean value is the gold repair.

Metric	Definition	Answers
Detection precision, recall, F1	Flagged cells against cells where dirty differs from clean	RQ1, RQ3
Error-type macro-F1	F1 over missing / pattern / rule / outlier on correctly detected errors	RQ1
Repair accuracy	Share of true errors that are flagged and repaired to exactly the clean value	RQ2
False-repair rate	Share of applied repairs that are wrong, including changes made to correct cells	RQ2
End-to-end correction precision, recall, F1	As in Baran: correctly fixed cells over all fixed cells (precision) and over all erroneous cells (recall)	RQ2 versus baselines
Calibration (ECE)	Expected calibration error, 10 bins, before and after temperature scaling	RQ2
Candidate recall and pass-through	Share of true errors kept by the cheap stage; share of all cells sent to the LLM	O4
Cost	GPU-seconds per 1,000 rows	O4
Downstream gap closed	(cleaned – dirty) / (gold-clean – dirty), for the same classifier and seeds	RQ4

Protocol. Learned methods and stochastic baselines are reported as mean ± standard deviation over four fixed seeds (42, 1, 2, 3); confidence intervals come from bootstrap resampling of test cells. Raha and Baran receive 20 labelled tuples per table, including unseen tables, while ContextGuard receives none on unseen tables. Baselines are rerun on our splits; published numbers are context only.

8.2 Baselines B1–B5

To ensure rigorous reproducibility, all baselines operate under a unified protocol.

B1

BASELINE NAME
Rule / Constraint Baseline

TYPE & MODEL
TANE (approximate functional dependency discovery) [13] feeding HoloClean [1]

INPUT PROVIDED
Dirty table only

SUPERVISION BUDGET
0 labels

OPERATIONAL DESCRIPTION
Removes the need for hand-written constraints, so the same pipeline runs on every table. TANE finds approximate dependencies X → Y on the dirty table; implausible ones are filtered out (key-like columns, large determinants, low support); each remaining dependency is written as a denial constraint (DC), and HoloClean detects the violating cells and repairs them through factor-graph inference. TANE only supplies the rules: detection and repair are HoloClean's. HoloClean was chosen over NADEEF, whose public code has not changed since 2016 and which runs through an interactive console, because it is implemented in Python under Apache-2.0 (last updated 2023) and runs without user labels. TANE is also the basis of detector D5, so the comparison shows what the language model adds over classical rule mining. The filter thresholds are fixed on the training tables in Milestone 2, never on test tables. The results published for HoloClean with hand-written constraints (Section 8.3.3) are context, not the target: they use different rules and the paper's own table versions.

B2

BASELINE NAME
Statistical Outlier Baseline

TYPE & MODEL
Unsupervised statistical detector (dBoost)

INPUT PROVIDED
Dirty table (numerical & categorical cols)

SUPERVISION BUDGET
0 labels

OPERATIONAL DESCRIPTION
Fits Gaussian, histogram, and mixture models per column. Flags statistical outliers; does not provide repairs.

B3

BASELINE NAME
Semi-Supervised Classical/ML

TYPE & MODEL
Raha (Detection) + Baran (Correction)

INPUT PROVIDED
Dirty table + active user query loop

SUPERVISION BUDGET
20 labeled tuples per table

OPERATIONAL DESCRIPTION
Standard Raha active sampling generates candidate features; trains Gradient/AdaBoost classifiers on 20 ground-truth labeled tuples to detect errors, then ensembles context correctors.

B4

BASELINE NAME
Zero-Shot Foundation LLM

TYPE & MODEL
Gemma 4 E4B-it (base weights, before fine-tuning)

INPUT PROVIDED
Serialized row context + target cell

SUPERVISION BUDGET
0 labels

OPERATIONAL DESCRIPTION
Prompted with row context and cell identifier in zero-shot format (thinking switched off). Returns structured JSON diagnosis and repair.

B5

BASELINE NAME
Few-Shot In-Context LLM

TYPE & MODEL
Gemma 4 E4B-it (base weights, before fine-tuning)

INPUT PROVIDED
3 in-context clean/dirty demonstration pairs

SUPERVISION BUDGET
0 table-specific labels (3 prompt exemplars)

OPERATIONAL DESCRIPTION
In-context demonstration exemplars illustrating contextual mismatch diagnosis and correction before prompting target cell.

8.3 Numbers to beat

All figures below were checked against the original papers, not secondary surveys. Raha's Table 5 values were checked against the authors' manuscript (MIT DSpace) and, according to the author of this section, against the ACM version.

8.3.1 Raha (SIGMOD 2019): Cell-Level Error Detection Baseline

- **Source:** Mahdavi et al. (*SIGMOD 2019*), Table 5 [4].
- **Experimental Protocol:** 20 labeled tuples budget ($\theta_{\text{Labels}} = 20$), Gradient Boosting meta-classifier, evaluated over 10 independent runs (reported std $< 1.0\%$).
- **Crucial Distinction:** Table 5 reports **cell-level** error detection metrics. Table 6 reports tuple-level metrics (e.g. Hospital tuple F1 = 0.79); only Table 5 is comparable with ContextGuard cell evaluations.

Benchmark Dataset	Shape (Rows × Cols)	Precision (P)	Recall (R)	F1-Score	Comparable to ContextGuard Data?
Hospital	1,000 × 19	0.94	0.59	**0.72**	**Yes:** Exactly identical 509 error cells as canonical Raha copy.
Flights	2,376 × 6	0.82	0.81	**0.81**	**Yes:** Identical 4,920 error cells.
Beers	2,410 × 10	0.99	0.99	**0.99**	**Yes:** Using canonical Raha repo version (4,362 error cells).
Rayyan	1,000 × 11	0.81	0.78	**0.79**	**Yes:** Identical 948 error cells.
Movies	7,390 × 17	0.85	0.88	**0.86**	**Yes:** Using canonical Raha repo version (7,675 error cells).
Standalone dBoost	Hospital / Flights / Beers	-	-	0.49 / 0.66 / 0.55	*Context baseline for standalone unsupervised statistical detection (B2).*

8.3.2 Baran (PVLDB 2020): Error Correction Baselines

- **Source:** Mahdavi & Abedjan (*PVLDB 2020*), Table 3 and Table 7 [5].
- **Experimental Protocol:** 20 labeled tuples ($\theta_{\text{Labels}} = 20$), AdaBoost classifier, mean over 10 independent runs.
- **Fair Comparison Note:** Table 3 assumes *perfect error detection* (ground truth error cells provided as input). Table 7 measures *end-to-end* error detection and repair (Raha detection $\rightarrow$ Baran repair), which is the valid benchmark to compare against ContextGuard.

Pipeline Configuration	Hospital (P / R / F1)	Flights (P / R / F1)	Beers (P / R / F1)	Rayyan (P / R / F1)	Tax (P / R / F1)
Baran (Perfect ED, Table 3)	0.88 / 0.86 / **0.87**	1.00 / 1.00 / **1.00**	0.91 / 0.89 / **0.90**	0.76 / 0.40 / **0.52**	0.84 / 0.78 / **0.81**
Baran + TL (Perfect ED, Table 3)	0.94 / 0.88 / **0.91**	1.00 / 1.00 / **1.00**	0.94 / 0.87 / **0.90**	0.80 / 0.44 / **0.57**	0.95 / 0.73 / **0.83**
HoloClean (Perfect ED, Table 3)	1.00 / 0.71 / **0.83**	0.89 / 0.67 / **0.76**	0.01 / 0.01 / **0.01**	0.00 / 0.00 / **0.00**	0.11 / 0.11 / **0.11**
Raha + Baran (End-to-End, Table 7) *(20 Detection + 20 Repair Labels)*	0.89 / 0.52 / **0.66**	0.88 / 0.53 / **0.66**	0.93 / 0.87 / **0.90**	0.50 / 0.27 / **0.35**	0.84 / 0.66 / **0.74**
Raha + Baran (In) (Table 7) *(20 Shared Labels)*	0.95 / 0.52 / **0.67**	0.84 / 0.56 / **0.67**	0.93 / 0.87 / **0.90**	0.44 / 0.21 / **0.28**	0.84 / 0.77 / **0.80**
Raha + Perfect Repair (Upper Bound)	0.98 / 0.58 / **0.73**	0.97 / 0.75 / **0.85**	0.98 / 1.00 / **0.99**	0.83 / 0.79 / **0.81**	0.97 / 0.98 / **0.97**

8.3.3 HoloClean (PVLDB 2017) & ZeroED (arXiv 2025) Cross-Check

- **HoloClean Verification:**
  - In Rekatsinas et al. (*PVLDB 2017*) [1], data characteristics are in **Table 1** (Hospital: 1,000 rows, 19 cols, 9 DCs, 6,140 noisy cells; Flights: 2,377 rows, 6 cols, 4 DCs, 11,180 noisy cells).
  - Cleaning effectiveness (precision, recall, F1) is in **Table 2**; Table 3 reports runtime:
  - Hospital ($\tau = 0.5$): Precision = **1.000**, Recall = **0.713**, F1 = **0.832**.
  - Flights ($\tau = 0.3$): Precision = **0.887**, Recall = **0.669**, F1 = **0.763**.
  - *Contextual Caveat:* Requires hand-coded denial constraints and relies on DeepDive factor graphs with domain pruning. These published numbers use hand-written constraints, so they are context for baseline B1 (which mines its constraints with TANE), not numbers it must match.
- **ZeroED Verification:**
  - Ni et al. (*arXiv 2025*) [16], Table III reports: Hospital F1 = **0.811**, Flights F1 = **0.722**, Beers F1 = **0.774**, Rayyan F1 = **0.732**, Billionaire F1 = **0.767**, Movies F1 = **0.765**.
  - *Discrepancy Analysis:* ZeroED numbers are **not directly comparable** as numbers to beat because: 1. ZeroED evaluated on altered table slices (e.g. Hospital 1,000 × 17 with 4.82% error rate vs standard 1,000 × 19 with 2.68% errors). 2. ZeroED restricted Raha to only 2 labeled tuples (resulting in an artificially suppressed Raha F1 of 0.125 on Hospital). 3. Results represent an average of only 3 stochastic runs. 4. ZeroED performs detection only, omitting error repair entirely.

8.4 Evaluation protocol

1. **Cell-Level Ground Truth Rule:** Following Raha and Baran, a cell (i, j) is considered an error if and only if its normalized dirty string differs from its normalized clean string:

$$\text{is_error}(i, j) \iff \text{normalize}(D_{i,j}) \neq \text{normalize}(C_{i,j})$$

   where `normalize(v)` performs:
   - HTML entity unescaping (`html.unescape`)

- Whitespace collapsing (`re.sub("[\t\n ]+", " ", text)` , as in Raha)
- Stripping of spaces, tabs and newlines (`.strip("\t\n ")`)

2. **Exact-Match Repair Rule:** A repair $\hat{R}_{i,j}$ for an error cell is scored correct if and only if:

$$\text{normalize}(\hat{R}_{i,j}) == \text{normalize}(C_{i,j})$$

   No partial credit, character edit distance discounts, or numeric tolerances are permitted.

3. **Core Evaluation Metrics:**
   - **Detection Precision (P_D):** $\frac{\text{Correctly Flagged Error Cells}}{\text{Total Flagged Cells}}$
   - **Detection Recall (R_D):** $\frac{\text{Correctly Flagged Error Cells}}{\text{Total Ground-Truth Error Cells}}$
   - **Detection F1 ($F1_D$):** $\frac{2 \cdot P_D \cdot R_D}{P_D + R_D}$
   - **Repair Precision (P_R):** $\frac{\text{Correctly Repaired Error Cells}}{\text{Total Cells Modified}}$
   - **Repair Recall (R_R):** $\frac{\text{Correctly Repaired Error Cells}}{\text{Total Ground-Truth Error Cells}}$
   - **Repair F1 ($F1_R$):** $\frac{2 \cdot P_R \cdot R_R}{P_R + R_R}$

4. **Data Splits and Label Isolation:**
   - **In-Domain Tables:** Evaluated on a held-out 15% row test split (seed 42).
   - **Unseen Tables:** Evaluated on 100% of held-out unseen tables (Restaurant, Marketing, Soccer, Billionaire, Clean-Me DE slice).
   - **Label Budget Fairness:** Baseline B3 receives 20 labeled tuples on all tables. ContextGuard receives **0 labels** on unseen tables. The 20 tuples labeled by B3 are strictly excluded from its scoring set.

5. **Replication & Seeds:** Stochastic runs and test samples use four fixed random seeds (42, 1, 2, 3), the same four samples as in the model comparison of Section 9. All metrics report **mean** $\pm$ **standard deviation**. With four seeds the standard deviation is only a rough measure of spread, so a difference smaller than one standard deviation is reported as no clear difference.

8.5 Experiments that isolate each component

Comparing the full system with the baselines shows whether it is better, not why. The experiments below change one thing at a time, with the same splits, seeds and metrics, so that each main idea is tested on its own. They are ordered by priority; combinations that do not test a main idea (for example every subset of detectors or of prompt blocks) are not planned.

#	Comparison	What differs	Question answered	Metrics	Priority
E1	Base model against the same model fine-tuned	Only the weights (the baseline is B4, zero-shot)	Does fine-tuning improve detection, repair and confidence?	Detection F1, repair accuracy, wrong-repair share, AUROC	1
E2	With and without context	Prompt: (a) cell and row only, (b) plus column profile, (c) plus dependency evidence and similar rows (full prompt)	Does information from the rest of the table help tell errors from unusual but valid values?	Same, per table and per error type	1 (a against c); b if time allows
E3	Candidate filtering alone, full pipeline, and the model on all cells	(a) cheap detectors only, (b) detectors followed by the model, (c) the model on every cell of a sample of rows	Does the model add enough over the filter to justify its cost, and how much does filtering lose and save?	Detection F1, candidate recall, share of cells sent to the model, GPU-seconds per 1,000 rows	1
E4	Repairs with and without the confidence gate	Apply every repair, or only those above the threshold	Does the gate make repairs safer, and at what loss of coverage?	Precision–coverage curve, wrong-repair share, ECE	2
E5	Training with and without synthetic examples	Synthetic share set to 0 or to the 20% cap	Do synthetic errors help on the types that are rare in real tables?	Detection F1 per error type	3, if time allows

E2 can be piloted early: the zero-shot model comparison scripts already build prompts, so a row-only prompt can be scored without any training. For the fine-tuned variants of E2, each prompt format gets its own training run. Each result is reported as mean ± standard deviation over the four seeds.

8.6 What the generalisation tests establish

Testing on a table that was not used in training is useful, but it shows less than the word "generalisation" suggests. The planned tests establish the following.

Test	What changes between training and test	A good result shows	It does not show
Held-out rows of the training tables (15%)	Only the rows; same tables, same kinds of error	The model learned the error patterns of these five tables	Anything about other tables
Unseen tables from other domains (Restaurant, Marketing, Soccer, Billionaire, Germany address slice)	The domain, the columns and values, and the process that made the errors; the four-type error taxonomy stays the same	Transfer across tables and domains, within the variety these five tables cover	Handling of *new kinds* of error; five tables are a small sample of domains
Unseen error types	Not covered by the unseen-table test, because every table uses the same four labelled types and the training data contains examples of all four	Only the exploratory leave-one-type-out check below	Cannot be claimed from the other two tests

As an exploratory check in Milestone 5, if time allows, one error type (for example outliers) is removed from the training data and the model is tested on that type. It is reported as exploratory, with its small sample stated. The final report will therefore say that the system generalises to *tables from other domains with the same four error types*, and will not say that it generalises to new kinds of error unless this check supports it.

8.7 How the results will be judged

The main goal is reliability of detection and repair (Section 6.1), so no single metric decides success. Every method, including the baselines, is shown on the same four axes:

1. **Detection quality:** precision, recall and F1 (macro-F1 over tables).
2. **Repair reliability:** the share of applied repairs that are wrong (false-repair rate).
3. **Coverage:** the share of true errors that are actually repaired at the chosen threshold.
4. **Cost:** the share of cells sent to the model and GPU-seconds per 1,000 rows.

Repair reliability and coverage pull in opposite directions, so they are compared on the precision–coverage curve in two ways: at the coverage the best baseline reaches (is our wrong-repair share lower?) and at the wrong-repair share of the best baseline (is our coverage higher?). A system that repairs few cells very reliably is reliable but not very useful for cleaning large tables, so coverage is always shown next to reliability. A system that repairs many cells but changes correct values often has coverage at the expense of reliability, and counts as worse on the main goal. The detected-but-not-repaired cells go to the review list, so recall of detection also shows how much work is left for a person.

How mixed results will be interpreted

These interpretations are fixed in advance.

Outcome	What it would mean	Consequence for the objectives
Detection improves, repair reliability does not	The model finds errors but fixes them badly	O1 met, O2 not met. The tool is positioned as detect-and-review, and only repairs above a high threshold are applied. Repair confidence is investigated
Repairs are very reliable but coverage is low	The gate is too strict, or the model is unsure on many cells	O2 met with limited usefulness. The threshold is tuned on validation data and the result is reported with its coverage
Coverage is high but wrong repairs are frequent	Reliability has been traded for coverage	Counts as a failure of the main goal, whatever the coverage. The threshold is raised
Detection and repair improve but cost is worse than the cheap baselines	Accuracy is bought with compute	O4 is judged against the model on every cell (B4, B5), the relevant comparison for cost. The result is stated as accuracy per GPU-second, with the trade-off explained
The filter removes many true errors (low candidate recall)	The recall ceiling limits the whole pipeline	Reported honestly; the candidate set is widened and E3 shows what that costs
Strong in-domain results, weaker on unseen tables	The model fits the training tables more than the task	Reported as limited transfer (Section 8.6); the claims are narrowed accordingly

Differences smaller than one standard deviation over the four seeds are reported as no clear difference. Exact numerical targets for each axis are fixed after the pilot runs in Milestone 3 and before the Milestone 5 evaluation, so that the rule for success is not chosen after seeing the results.

9 Feasibility and plan

Compute. The pipeline is designed to run on a single GPU. In the zero-shot pilot the three open models needed 14 to 23 GiB of GPU memory in bf16 (Gemma 4 E4B-it about 19 GiB), so LoRA fine-tuning of an ~8B model in bf16 fits on one modern GPU.

Data readiness. The five core benchmark tables hold **18,414 verified error cells** (Hospital 509, Flights 4,920, Beers 4,362, Rayyan 948, Movies 7,675), and their ground truth has been checked against the original Raha files. Further tables for training (Tax, Inpatient, Facilities), unseen-table testing (Restaurant, Marketing, Soccer, Billionaire, Clean Me If You Can) and the downstream test (Credit) are listed in Section 7. Raw data stays out of the public repository; we provide download scripts and cite every source.

Feasibility evidence. A zero-shot pilot of three open models on three random samples of 1,500 cells from the five core tables produced valid structured answers in over 99% of cases at about 0.2–0.3 seconds per cell on one GPU, with an error-detection AUROC of 0.78–0.86 before any fine-tuning. The task is therefore learnable with the available hardware; fine-tuning and the confidence gate are what the next milestones add.

Milestone	Due	Work
M2 Dataset preparation	29 Oct	Verified sources, splits with leakage checks, data card, first instruction–response examples
M3 Model architecture	5 Nov	Pipeline end to end on a subset, input formats, justification, loss and metrics
M4 Model training	19 Nov	Fine-tuning runs, component experiments E1 to E3 (Section 8.5), hyper-parameter experiments with four seeds
M5 Evaluation and analysis	26 Nov	Baseline comparison, precision–coverage curve, unseen tables, downstream experiment, error analysis
M6 Deployment and documentation	10 Dec	Gradio demo and API, model and data cards, final report

Risk	Effect	Mitigation
Candidate stage misses many errors	Recall ceiling for the whole system	Lower detector thresholds; add "missed errors" to training; report the ceiling honestly
Model over-flags rare but correct values	Low precision	Hard negatives in training; calibrate on validation data; raise the flag threshold
Unclear licences (TableEG states none; Clean-Me data is ODbL share-alike)	Reproducibility and demo risk	Keep raw data out of the repo; cite each original source; download scripts only
Credit's "clean" income values were imputed, and its dirty–clean gap may be small	RQ4 inconclusive	State the caveat; repeat the downstream test on Beers (ABV)
Tables with little redundancy or many errors give the model weak or misleading context	Lower precision on those tables	Report per table; compare with and without context (E2); expect smaller gains there
A new model family	Library problems	Keep the pipeline model-agnostic so another model can be swapped in

A Reference material for the cheap detectors (D3–D6)

ContextGuard's initial stage applies fast, lightweight classical detectors to surface candidate erroneous cells before invoking the LLM. Each detector is formally grounded in established statistical and algorithmic literature:

D3: Robust Z-Score with Median Absolute Deviation (MAD)

- **Reference:** Iglewicz, B., & Hoaglin, D. C. (1993). *How to Detect and Handle Outliers* (ASQC Basic References in Quality Control, Vol. 16) [9].
- **Formulation:** Modified Z-score $M_i = \frac{0.6745 \cdot (x_i - \tilde{x})}{\text{MAD}}$, where $\text{MAD} = \text{median}(|x_i - \tilde{x}|)$ and threshold $|M_i| > 3.5$.
- **What it catches:** Univariate continuous numerical anomalies and extreme values that deviate sharply from the median, immune to masking effects that distort sample mean and standard deviation.
- **What it misses:** Valid numbers that are contextually invalid given neighboring attributes (e.g., Patient Age = 32 with Procedure = Pediatric Heart Surgery, or Zip = 90210 with City = Chicago).

D4: Isolation Forest & Local Outlier Factor (LOF)

- **Reference:** Liu, F. T., Ting, K. M., & Zhou, Z.-H. (ICDM 2008). *Isolation Forest* [10]; Breunig, M. M., et al. (SIGMOD 2000). *LOF: Identifying Density-Based Local Outliers* [11].
- **Formulation:** Multi-dimensional tree isolation depth and local kernel density ratios comparing local reachability density against k-nearest neighbors.
- **What it catches:** Multi-attribute numerical outliers and isolated cluster points in continuous feature spaces (e.g., extreme income-to-debt ratios in Credit).
- **What it misses:** Categorical, syntactic, and semantic typographical errors (e.g., state name misspellings, mismatched string codes, missing strings).

D5: Approximate Functional Dependencies (AFDs)

- **Reference:** Kivinen, J., & Mannila, H. (1995). *Approximate inference of functional dependencies from relations* [12]; Huhtala, Y., et al. (1999). *TANE: An efficient algorithm for discovering functional and approximate dependencies* [13].
- **Formulation:** Discovers column dependencies $X \to Y$ satisfying the g_3 error measure:

$$g_3(X \to Y) = 1 - \frac{\sum_{c \in \pi_X} \max_y |\{t \in c : t[Y] = y\}|}{|r|} \leq \epsilon$$

  where π_X are the groups of rows sharing the same X value and ϵ is the allowed violation tolerance: g_3 is the smallest fraction of rows that must be removed for $X \to Y$ to hold exactly.
- **What it catches:** Determinate attribute violations where one column almost strictly determines another across the dataset (e.g., `ZipCode -> City` or `Hospital_ID -> Hospital_Name`).
- **What it misses:** Errors in non-determinate columns (independent attributes), tables with low redundancy, or errors where both the determinant and dependent attribute are simultaneously corrupted.

D6: Edit-Distance Near-Duplicates & Lexical Variation

- **Reference:** Levenshtein, V. I. (1966). *Binary codes capable of correcting deletions, insertions, and reversals* [14]; Navarro, G. (2001). *A guided tour to approximate string matching* [15].
- **Formulation:** Normalized Levenshtein distance $\mathrm{NLD}(s_1, s_2) = \frac{\mathrm{dist}_L(s_1, s_2)}{\max(|s_1|, |s_2|)} \leq \theta_{\text{edit}}$ between low-frequency tokens and high-frequency active-domain cluster centroids.
- **What it catches:** Typographical slips, character swaps, dropped characters, and optical character recognition errors in categorical strings (e.g., `Scaramento, CA` vs `Sacramento, CA`).
- **What it misses:** Semantic swaps where an erroneous entry is a valid, correctly spelled token in the active domain (e.g., `Male` swapped to `Female`, or city `Boston` swapped to valid city `Chicago`).

B Installation check for Raha (head start for Milestone 3)

To verify baseline operational feasibility and prevent dependencies from blocking future milestones:

- **Environment:** Python `3.11.3`, `pandas 2.2.3` (verified `< 3.0.0` compatibility), `scikit-learn 1.8.0`, `numpy 1.26.4`.
- **Raha Release:** Installed `raha==1.26` from PyPI.

- **Identified OS Compatibility Issue & Fix:**
  - *Observation:* On Windows OS, Raha's multiprocessing runner throws `PermissionError: [WinError 32]` during dBoost temporary CSV cleanup (`os.remove(dataset_path)`), as file handles remain locked across worker processes.
  - *Mitigation:* Implemented safe file deletion exception handling in `scripts/run_raha_baseline.py` .
- **Hospital Smoke Test Execution Result:**
  - Benchmark run: `scripts/run_raha_baseline.py` with 20 labeled tuples on `hospital` (Seed 42).
  - Evaluated on canonical 509 ground-truth error cells.
  - **Empirical Metrics:**
  - **Precision:** `0.7149`
  - **Recall:** `0.6110`
  - **F1-Score:** `0.6589`
  - *Finding:* Consistent with single-run variability: in an independent check, three Raha runs on Hospital gave F1 between 0.57 and 0.81, while the paper reports 0.72 averaged over 10 runs. Confirms the imperative of evaluating across 10 randomized seeds. The raw console output is in `outputs/logs/raha_hospital_smoke_test.txt` ; that run reused saved strategy results from an earlier run on the same machine ("I just load strategies' results as they have already been run").

C Team contributions for Milestone 1

Member	Contribution	Worklog
Sidhaarth Shree	Problem framing, objectives and metrics, LLM literature, contribution, feasibility; model comparison; merge and submission	`Milestone1_Worklog.md`
Jashan	Audit of the nine benchmark tables; Section 7 (benchmark tables)	same
Sahilpreet	Download and audit of the external datasets; download log	same
Aaradhya	Classical and learning-based literature, detector references, numbers to beat, baselines and protocol, tools review, Raha check	same

References

1. **Rekatsinas, T., Chu, X., Ilyas, I. F., & Ré, C.** (2017). HoloClean: Holistic Data Repairs with Probabilistic Inference. *Proceedings of the VLDB Endowment (PVLDB)*, 10(11), 1190–1201. doi:10.14778/3137628.3137631.
2. **Dallachiesa, M., Ebaid, A., Eldawy, A., Elmagarmid, A., Ilyas, I. F., Ouzzani, M., & Tang, N.** (2013). NADEEF: A Commodity Data Cleaning System. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 541–552. doi:10.1145/2463676.2465327.
3. **Papenbrock, T., & Naumann, F.** (2016). A Hybrid Approach to Functional Dependency Discovery. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 821–833. doi:10.1145/2882903.2915203.
4. **Mahdavi, M., Abedjan, Z., Castro Fernandez, R., Madden, S., Ouzzani, M., Stonebraker, M., & Tang, N.** (2019). Raha: A Configuration-Free Error Detection System. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 865–882. doi:10.1145/3299869.3324956.
5. **Mahdavi, M., & Abedjan, Z.** (2020). Baran: Effective Error Correction via a Unified Context Representation and Transfer Learning. *Proceedings of the VLDB Endowment (PVLDB)*, 13(11), 1948–1961. doi:10.14778/3407790.3407801.
6. **Qin, J., Huang, S., Wang, Y., Zhu, J., Zhang, Y., Miao, Y., Mao, R., Onizuka, M., & Xiao, C.** (2024). BClean: A Bayesian Data Cleaning System. *IEEE International Conference on Data Engineering (ICDE)*, 3407–3420. doi:10.1109/ICDE60146.2024.00263; preprint arXiv:2311.06517.
7. **Li, P., Rao, X., Blase, J., Zhang, Y., Chu, X., & Zhang, C.** (2021). CleanML: A Study for Evaluating the Impact of Data Cleaning on ML Classification Tasks. *IEEE 37th International Conference on Data Engineering (ICDE)*, 13–24. doi:10.1109/ICDE51399.2021.00009.
8. **Krishnan, S., Wang, J., Wu, E., Franklin, M. J., & Goldberg, K.** (2016). ActiveClean: Interactive Data Cleaning for Statistical Modeling. *Proceedings of the VLDB Endowment (PVLDB)*, 9(12), 948–959. doi:10.14778/2994509.2994514.
9. **Iglewicz, B., & Hoaglin, D. C.** (1993). *How to Detect and Handle Outliers*. ASQC Basic References in Quality Control, Vol. 16. ISBN 0-87389-247-9.
10. **Liu, F. T., Ting, K. M., & Zhou, Z.-H.** (2008). Isolation Forest. *IEEE International Conference on Data Mining (ICDM)*, 413–422. doi:10.1109/ICDM.2008.17.
11. **Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J.** (2000). LOF: Identifying Density-Based Local Outliers. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 93–104. doi:10.1145/342009.335388.
12. **Kivinen, J., & Mannila, H.** (1995). Approximate Inference of Functional Dependencies from Relations. *Theoretical Computer Science*, 149(1), 129–149. doi:10.1016/0304-3975(95)00028-U.
13. **Huhtala, Y., Kärkkäinen, J., Porkka, P., & Toivonen, H.** (1999). TANE: An Efficient Algorithm for Discovering Functional and Approximate Dependencies. *The Computer Journal*, 42(2), 100–111. doi:10.1093/comjnl/42.2.100.
14. **Levenshtein, V. I.** (1966). Binary Codes Capable of Correcting Deletions, Insertions, and Reversals. *Soviet Physics Doklady*, 10(8), 707–710.
15. **Navarro, G.** (2001). A Guided Tour to Approximate String Matching. *ACM Computing Surveys*, 33(1), 31–88. doi:10.1145/375360.375365.

16. **Ni, W., Zhang, K., Miao, X., Zhao, X., Wu, Y., Wang, Y., & Yin, J.** (2025). ZeroED: Hybrid Zero-shot Error Detection through Large Language Model Reasoning. *arXiv preprint arXiv:2504.05345*. arxiv.org/abs/2504.05345.
17. Bendinelli et al. *Exploring LLM Agents for Cleaning Tabular Machine Learning Datasets.* ICLR 2025 Workshop. arXiv:2503.06664.
18. Zhang et al. *Jellyfish: Instruction-Tuning Local Large Language Models for Data Preprocessing.* EMNLP 2024. aclanthology.org/2024.emnlp-main.497.
19. Li et al. *Table-GPT: Table-tuned GPT for Diverse Table Tasks.* SIGMOD 2024. arXiv:2310.09263.
20. Zhang, Huang and Wu. *Data Cleaning Using Large Language Models (Cocoon).* arXiv:2410.15547, 2024.
21. Ni et al. *IterClean: An Iterative Data Cleaning Framework with Large Language Models.* ACM TURC 2024. doi:10.1145/3674399.3674436.
22. Wang et al. *Ensembling LLM-Induced Decision Trees for Explainable and Robust Error Detection (TreeED/ForestED).* KDD 2026. arXiv:2512.07246.
23. Liu et al. *Towards Practical Benchmarking of Data Cleaning Techniques: On Generating Authentic Errors via Large Language Models (TableEG).* arXiv:2507.10934.
24. Chen et al. *Empowering Tabular Data Preparation with Language Models: Why and How?* arXiv:2508.01556, 2025.